Automatic Topic Tagging for Student Notes

Project report, reproducible notebook, evaluation, and interactive prototype

Prepared by: R. Shradha Sri

Project question: Can a text-processing system assign useful topic tags to student notes while allowing users to correct mistakes?

Submission scope: taxonomy, synthetic labelled dataset, preprocessing, keyword baseline, TF-IDF multi-label classifier, evaluation and error analysis, prototype, privacy safeguards, user guide, and presentation outline.

Important: All included notes are synthetic and created for this project. The results demonstrate a reproducible workflow on a small classroom dataset; they are not evidence of production-level accuracy.

Executive Summary

This project builds a transparent student-note tagging prototype. A note can receive multiple topic labels because a single paragraph may discuss several subjects. The system compares an explainable keyword/rule baseline with a TF-IDF text representation and a one-vs-rest logistic-regression classifier. The notebook records the data split, metrics, predictions, and limitations. A Streamlit interface demonstrates tag suggestions, editable tags, and keyword search.

The design prioritizes human control: model output is a suggestion, users can add or remove tags, and the system should not be used for grading or other high-stakes decisions. The included dataset contains 72 synthetic notes and 13 topic labels. The train/test split is deterministic (random seed 42), with 25% held out for evaluation.

1. Problem Statement and Objectives

Students collect notes across subjects, but finding a specific concept later can be difficult when files have inconsistent names or no tags. Manual tagging is flexible but takes time. Automatic tagging can reduce repetitive work, provided that it handles overlapping topics, short notes, technical vocabulary, and user corrections.

- Define a compact taxonomy with descriptions and multi-label examples.
- Prepare and inspect synthetic educational notes with manually assigned tags.
- Preserve meaningful technical tokens during text preprocessing.
- Compare a transparent rule-based baseline with TF-IDF + logistic regression.
- Measure precision, recall and F1, then inspect incorrect and missing tags.
- Provide a searchable interface with visible, editable suggestions.
- Document privacy, permissions, limitations, and reproducibility.

2. Scope, Assumptions, and Tools

Scope: English-language educational notes across introductory computing, data, and responsible-AI topics. The prototype is designed for classroom demonstration and local execution.

Tool	Purpose
Python	Data handling, model training, evaluation and prototype logic
pandas	CSV loading, cleaning and summary tables
scikit-learn	TF-IDF vectorization, multi-label classifier, train/test split and metrics
Jupyter / Google Colab	Reproducible notebook execution

Tool	Purpose
Streamlit	Interactive tagging and search interface
CSV files	Portable dataset, taxonomy and evaluation output

3. Data Source, Permission, and Labelled Dataset

The project uses synthetic text written specifically for this demonstration, rather than copied student records or third-party educational passages. This avoids exposing personal information and avoids dependence on external text reuse permissions. If the project is extended with public/open material, record the source URL, author, licence, retrieval date, and any attribution requirements before reuse.

The dataset has 72 notes and 13 labels. Each row contains a note ID, note text, one or more manually assigned tags, and a semicolon-separated tag field for simple CSV interoperability. Multi-label notes are intentional.

Dataset field	Description
note_id	Unique synthetic note identifier
note_text	Short educational note or code-oriented explanation
tags	List of manually assigned topic labels
tags_string	Same labels separated by semicolons for CSV use

4. Topic Taxonomy

A taxonomy is a controlled vocabulary. Clear definitions reduce inconsistent labelling and help evaluators decide whether a predicted tag is appropriate. Related labels are retained separately because the project explicitly tests overlapping topics.

Tag	Description
Programming	Code, language syntax, variables, loops, functions, and implementation concepts.
Algorithms	Problem-solving procedures, complexity, searching, sorting, recursion, and algorithm design.
Data Structures	Ways to organize data, such as arrays, stacks, queues, linked lists, trees, and hash tables.
Databases	Relational data, SQL, tables, keys, joins, transactions, and database design.
Machine Learning	Learning from data, model training, prediction, generalization, and ML evaluation.
Statistics	Descriptive statistics, probability, distributions, metrics, and statistical reasoning.
Web Development	Web pages, HTML, CSS, JavaScript, HTTP, APIs, and responsive interfaces.
Software Engineering	Requirements, testing, version control, reviews, integration, and software development practices.
Cybersecurity	Protecting systems and information, authentication, encryption, phishing, and access control.
Artificial Intelligence	AI methods and systems, including neural networks, language processing, and generative AI.
Information Retrieval	Searching, indexing, ranking, text vectors, TF-IDF, and similarity-based retrieval.
Data Analysis	Preparing, exploring, summarizing, and visualizing data using tables, metrics, and charts.
Responsible AI	Privacy, fairness, transparency, licensing, limitations, and human oversight of AI systems.

5. Data Preparation and Preprocessing

- Load the CSV with pandas and confirm required columns and non-empty text.
- Preserve headings and technical terms; avoid aggressive stop-word removal that may remove meaning.
- Normalize case and strip accents in the TF-IDF vectorizer.
- Use unigram and bigram features so phrases such as “binary search” and “primary key” can be represented.

- Retain common code and technical tokens with a token pattern that permits characters such as plus signs, underscores, dots, and hyphens.
- Split the dataset into training and held-out test sets before fitting the vectorizer to avoid test-set vocabulary leakage.
- Encode tags using a multi-label indicator matrix; one note can have several positive labels.

Potential future cleaning steps include removing accidental duplicate notes, detecting empty/near-empty notes, normalizing whitespace, and masking personal identifiers. Code-heavy notes should be preserved as text and tested separately rather than stripped of punctuation.

6. Method A — Keyword/Rule Baseline

The baseline maps each tag to a set of indicative terms or phrases. A note receives a tag when one or more of that tag's rules match. This method is fast, understandable, and easy to audit, but it depends on manually maintained terms.

Example: "A stack follows last-in first-out order" can trigger Data Structures. "Binary search halves the search range" can trigger Algorithms and Data Structures depending on the rule set.

- Strengths: interpretable, fast, no model-training requirement, easy to edit.
- Weaknesses: synonyms may be missed; ambiguous terms can create false positives; rule coverage is uneven.
- Use as a baseline, not as unquestionable ground truth.

7. Method B — TF-IDF + Multi-label Logistic Regression

TF-IDF represents each note using weighted word and phrase features. Terms that occur in many notes are down-weighted relative to terms that are distinctive to a note. A one-vs-rest logistic-regression model learns a separate binary decision function for each tag. This approach is a suitable compact baseline for small text datasets and supports multiple tags per note.

Configuration: lowercase text, Unicode accent stripping, unigram/bigram range (1,2), sublinear term frequency, logistic regression with a high iteration limit, and balanced class weights. The train/test split uses 75% training and 25% testing with random_state=42.

The demonstration uses model decision scores to rank suggestions in the interface. Thresholds should be tuned on a validation set before real use. If no tag crosses the threshold, the interface can display a small number of top candidates clearly marked as suggestions.

8. Evaluation Protocol and Results

The held-out set is used only for evaluation. Precision measures how many assigned tags are correct; recall measures how many correct tags are found; F1 balances precision and recall. Micro averages pool label decisions, while macro averages weight each label equally. Exact-match accuracy counts a note as correct only when its full predicted tag set matches the reference set.

Model	Micro P	Micro R	Micro F1	Macro P	Macro R	Macro F1	Exact match
Keyword/rule baseline	0.815	0.815	0.815	0.667	0.674	0.659	0.556
TF-IDF + One-vs-Rest Logistic Regression	0.875	0.259	0.400	0.308	0.176	0.207	0.111

The exact scores are saved in evaluation_results.csv and can be regenerated by running the notebook. Since the dataset is small, scores may change substantially if the sample or split changes. Do not interpret this single split as a robust estimate of real-world performance.

9. Error Analysis and Overlapping Topics

The file test_predictions_error_analysis.csv lists each test note, its reference tags, the baseline output, and the classifier output. Review false positives (suggested tag not in the reference set) and false negatives (reference tag not predicted).

- Programming vs Algorithms: code examples may describe an algorithm as well as language syntax.
- Machine Learning vs Statistics: precision, recall, and evaluation metrics appear in both domains.

- Information Retrieval vs Machine Learning: TF-IDF and vector representations can be relevant to both.
- Short notes: a short phrase may not contain enough context to disambiguate a label.
- Code-heavy notes: punctuation and identifiers are meaningful, but a bag-of-words model does not understand code execution.
- Unfamiliar terminology: new abbreviations, misspellings, and course-specific vocabulary may be missed.

Recommended review: inspect the most frequent false positives and false negatives, update taxonomy definitions and training examples, and re-evaluate on a fresh held-out set. Do not tune on the final test set repeatedly.

10. Short Notes, Code-heavy Notes, and Novel Terms

Test type	Example	Expected handling / risk
Short note	“Queue uses FIFO.”	Likely Data Structures; low context can reduce confidence.
Code-heavy	`for i in range(5): print(i)`	Preserve syntax and tokens; likely Programming.
Cross-topic	“TF-IDF features feed a classifier.”	Information Retrieval and Machine Learning may both apply.
Unfamiliar term	A new course-specific acronym	Suggest cautiously; allow search/manual correction.
Empty input	Whitespace only	Show validation message; do not predict.

11. Interactive Prototype and User Correction

The supplied app.py is a Streamlit prototype. It has a note-entry area, a “Suggest tags” action, a multi-select control to add or remove tags, and a search box for the synthetic dataset. The final user-selected tags are displayed separately from the initial suggestions.

- Input: student enters a note.
- Suggestion: the model ranks likely tags.
- Correction: the student adds or removes tags using the multi-select control.
- Search: the student searches example notes by a word or phrase.
- Human authority: edits override model suggestions; no automatic grades or penalties are produced.

12. Privacy, Safety, and Responsible Use

- Data minimization: collect only note text needed for tagging.
- Consent and permissions: use synthetic, self-authored, or properly licensed data; document permissions.
- Personal information: do not enter names, student IDs, grades, contact details, health information, or confidential course material into a shared demo.
- Local processing: the included Streamlit app runs locally; do not expose it publicly without access controls and a privacy review.
- Human control: all tags are editable, and the user may leave a note untagged.
- Transparency: disclose that suggestions are machine-generated and may be wrong.
- Fairness: evaluate performance across subjects, note lengths, language varieties, and writing styles.
- Retention: define how notes and corrections are stored; this prototype does not require saving user-entered notes to a database.
- Purpose limitation: do not use tags to infer ability, rank students, grade work, or make high-impact decisions.

13. Reproducibility and Installation

The project files are designed to run together in one folder. Python 3.10 or newer is recommended.

- Install dependencies: pip install -r requirements.txt

- Notebook: open capstone28_tagging_notebook.ipynb in Jupyter or upload it with the CSV files to Google Colab.
- Interface: run streamlit run app.py from the project folder.
- Reproducibility: use random_state=42, retain the dataset and taxonomy versions, and record package versions when reporting results.
- Evaluation files: evaluation_results.csv and test_predictions_error_analysis.csv contain the generated outputs from the fixed split.

14. Limitations and Future Improvements

- Small synthetic dataset: limited vocabulary and label variety; performance does not generalize automatically.
- Imbalanced labels: some topics have fewer examples, making macro metrics unstable.
- Linear text features: TF-IDF does not deeply understand context or semantic similarity.
- Thresholds: current demonstration uses decision-score ranking; validation-based threshold calibration is needed.
- Future work: add consented real notes, use a separate validation set, compare a pretrained embedding model, collect user corrections with consent, and monitor drift.
- Accessibility: support keyboard navigation, clear labels, and readable contrast in future interface refinement.

15. Conclusion

The project demonstrates an end-to-end, multi-label topic-tagging workflow for student notes: a defined taxonomy, synthetic labelled data, careful preprocessing, a rule baseline, TF-IDF classification, evaluation, error inspection, and a correction-friendly interface. Its central design principle is that tags are suggestions. Student edits override model output, and responsible deployment requires permissioned data, transparent limitations, and further evaluation.

Appendix A — Sample labelled examples

ID	Note excerpt	Reference tags
N001	A variable stores a value in memory. Python uses assignment with the equals sign, and variable names…	Programming
N002	A for loop repeats a block of code for each item in a sequence. Indentation defines the loop body in…	Programming
N003	Functions group reusable instructions. Parameters receive input values and return sends a result to …	Programming
N004	An algorithm is a finite sequence of steps used to solve a problem. Pseudocode describes logic befor…	Programming;Algorithms
N005	Binary search finds a target in a sorted array by repeatedly checking the middle element and halving…	Algorithms;Data Structures
N006	A stack follows last-in first-out order. Push adds an item and pop removes the most recent item.	Data Structures
N007	A queue follows first-in first-out order. Enqueue inserts at the rear and dequeue removes from the f…	Data Structures
N008	A linked list stores nodes connected by references. Each node contains data and a pointer to the nex…	Data Structures
N009	Big O notation describes how an algorithm's time or space requirement grows with input size.	Algorithms
N010	Merge sort divides an array, sorts each half recursively, and merges the sorted halves in O(n log n)…	Algorithms;Programming
N011	A database table contains rows and columns. A primary key uniquely identifies each record.	Databases
N012	SQL SELECT retrieves columns from a table, while WHERE filters rows that meet a condition.	Databases

Appendix B — Five-point user checklist

- Use only self-authored, synthetic, or permissioned/licensed notes.
- Remove personal identifiers and confidential information before processing.
- Review suggested tags before relying on them; correct or remove errors.
- Do not use the prototype for grading, student ranking, or high-stakes decisions.
- Document dataset source, licence/permission, model settings, evaluation results, and known limitations.

Appendix C — Final presentation outline

Slide	Content
1. Title	— Automatic Topic Tagging for Student Notes
2. Problem	Notes are hard to find when tags are missing or inconsistent.
3. Goal	Suggest multiple topic tags while keeping users in control.
4. Dataset & taxonomy	72 synthetic notes; 13 defined tags; multi-label annotations.
5. Method	Keyword rules vs TF-IDF + one-vs-rest logistic regression.
6. Evaluation	Micro/macro precision, recall, F1 and exact match; discuss small-data limits.
7. Demo	Enter a note, review suggestions, edit tags, and search examples.
8. Responsible AI	Synthetic data, privacy safeguards, user correction, no high-stakes use.
9. Conclusion	Useful starting point; improve with more representative labelled data.

End of report. Supporting files are included alongside this PDF in the project bundle.